

SMLDS - Internal Exam Study Notes

Question Bank 02 · Section A (2 marks, 4 points each) · Section B (5 marks, 10 points each)

SECTION A - 2 Marks Each

Q1. What is the difference between precision and recall? Mention one situation where recall is preferred over precision.

Point	Precision	Recall
1. Definition	Precision is the fraction of the instances predicted as positive that are actually positive, i.e. $TP / (TP + FP)$.	Recall is the fraction of all actual positive instances that the model correctly identifies, i.e. $TP / (TP + FN)$.
2. What it tells us	It tells us how reliable the model's positive predictions are.	It tells us how many of the real positive cases the model is able to find.
3. Error it penalises	It is reduced by false positives, so it penalises false alarms.	It is reduced by false negatives, so it penalises missed cases.
4. When it matters	It is important when a false alarm is costly, e.g. spam filtering.	It is important when missing a positive is costly, e.g. disease detection.

Recall is preferred when:

Cancer screening – missing a sick patient (FN) is far worse than a false alarm.

Example: out of 100 cancer patients the model detects 95, so recall = 0.95.

Q2. A classification model produces TP = 40, FP = 10, FN = 5, TN = 45. Calculate the precision.

Given	TP = 40, FP = 10, FN = 5, TN = 45
Formula	Precision = $TP / (TP + FP)$
Substitution	Precision = $40 / (40 + 10) = 40 / 50$
Result	Precision = 0.80 (80%)
Conclusion	80% of the instances predicted positive are actually positive.
Extra check	Recall = $40/45 = 0.889$; Accuracy = $85/100 = 0.85$; $F1 \approx 0.842$

Q3. What is SMOTE? Why is it used when handling an imbalanced classification dataset?

1. SMOTE = Synthetic Minority Over-sampling Technique.

2. For a minority point it picks one of its k nearest minority neighbours and creates a synthetic point on the line between them: $x_{\text{new}} = x + \lambda(x_{\text{neighbour}} - x)$, $\lambda \in [0, 1]$.

3. Used because imbalanced data biases the model towards the majority class – accuracy looks high but minority recall is poor.

4. Unlike random duplication it creates new points, balancing classes and reducing overfitting. Apply on the training set only.

Example: 950 genuine and 50 fraud transactions – SMOTE generates synthetic fraud samples until both classes have about 950.

Q4. What is the kernel trick in SVM? State its purpose.

1. Some data is not linearly separable in its original space.

2. The kernel trick computes the dot product $\phi(x) \cdot \phi(x')$ in a higher-dimensional space directly as $K(x, x')$, without computing the mapping ϕ .

3. Purpose: lets SVM draw non-linear decision boundaries efficiently and cheaply.

4. Common kernels: linear, polynomial $(x^T x' + c)^d$, RBF $\exp(-\gamma \|x - x'\|^2)$, sigmoid.

Example: one class forms a ring around another in 2-D; the RBF kernel lifts the points to a higher dimension where a plane separates them.

Q5. What is the purpose of explained variance in PCA? How does it help in selecting the number of principal components?

1. Explained variance = proportion of total variance captured by each principal component: $\lambda_i / \sum \lambda$.
2. It shows how much information each component retains.
3. Plot cumulative explained variance (scree plot); pick the smallest number of components reaching a threshold (90-95%) or use the elbow.
4. This keeps most information while reducing dimensions – simpler, faster, less noisy model.
Example: PC1 = 60%, PC2 = 25%, PC3 = 10%, PC4 = 5% → cumulative 60, 85, 95, 100. To keep 95% choose 3 components.

Q6. Mean = 50, SD = 5. Calculate the Z-score of an observation of 65. Is it an outlier using $|Z| > 3$?

Given	$\mu = 50, \sigma = 5, x = 65$
Formula	$Z = (x - \mu) / \sigma$
Substitution	$Z = (65 - 50) / 5 = 15 / 5$
Result	Z = 3
Conclusion	Criterion is $ Z > 3$. Here $ Z = 3$, which is not greater than 3, so the observation is not an outlier (exactly on the borderline).

Q7. What is the main difference between PCA and LDA in terms of whether they use class labels?

Point	PCA	LDA
1. Use of class labels	PCA does not use class labels; it is an unsupervised technique.	LDA uses the class labels; it is a supervised technique.
2. Objective	PCA finds the directions along which the variance of the data is maximum.	LDA finds the directions that maximise the separation between the classes.
3. Method	PCA takes the eigenvectors of the covariance matrix of the data.	LDA maximises the ratio of between-class scatter to within-class scatter.
4. Number of components	PCA can give as many components as there are features.	LDA can give at most (number of classes – 1) components.

Example: for 3 flower species LDA gives at most 2 components that separate the species; PCA gives directions of maximum spread regardless of species.

Q8. What is the silhouette score? What does a high score indicate?

1. For each point $s = (b - a) / \max(a, b)$; a = mean distance to own cluster, b = mean distance to the nearest other cluster.
2. Value ranges from -1 to $+1$.
3. A high score (close to $+1$) means tight clusters, well separated from others – good clustering quality.
4. Near 0 = overlapping clusters; negative = likely wrong assignment. Also used to choose the best k .

Example: $a = 2, b = 8 \rightarrow s = (8 - 2)/8 = 0.75$, a well-separated point.

Q9. Differentiate between AIC and BIC. Mention one key difference in how they penalise complexity.

Point	AIC	BIC
1. Full form	AIC stands for Akaike Information Criterion.	BIC stands for Bayesian Information Criterion.
2. Formula	$AIC = 2k - 2\ln(L)$.	$BIC = k \cdot \ln(n) - 2\ln(L)$.

Point	AIC	BIC
3. Penalty on complexity	AIC charges a fixed penalty of 2 for every parameter added, whatever the sample size.	BIC charges a penalty of $\ln(n)$ per parameter, which increases as the sample size n grows.
4. Model preference	AIC tends to select a slightly more complex model and is better for prediction.	BIC penalises complexity more heavily (for $n > 7$), so it selects simpler models and is better for finding the true model.

Lower value is better for both (k = parameters, n = sample size, L = likelihood).

Example: $n = 100 \rightarrow \ln(100) \approx 4.6$, so BIC charges ≈ 4.6 per parameter vs AIC's 2; BIC rejects extra variables more readily.

Q10. High training accuracy but much lower validation accuracy - identify the issue using bias-variance trade-off.

1. Likely issue: **overfitting** - low bias, **high variance**.
2. The model memorised noise and specifics of the training data, so it generalises poorly.
3. The gap between training and validation accuracy is the sign of high variance.
4. Fixes: regularisation (L1/L2), more data, simpler model, pruning, dropout, cross-validation, early stopping.

Example: training accuracy 98%, validation accuracy 70% $\rightarrow$ overfitting.

SECTION B - 5 Marks Each (10 points each)

B1. Fraud detection on an imbalanced dataset. a) Why can accuracy mislead? b) How do precision, recall and F1 help? c) Which metric to prioritise for detecting as many frauds as possible?

a) Why accuracy misleads

1. Fraud may be under 1% of all transactions.
2. A useless model that predicts "genuine" for everything still scores $\approx 99\%$ accuracy.
3. Accuracy counts all correct predictions equally, so the majority class dominates and it says nothing about fraud detection.

Example: 990 genuine + 10 fraud: always predicting "genuine" gives 99% accuracy but 0% recall.

b) Precision, recall, F1

4. **Precision** = $TP/(TP+FP)$: how many flagged transactions are truly fraud; low precision = many genuine customers disturbed.
5. **Recall** = $TP/(TP+FN)$: how many real frauds are caught; exposes the misses accuracy hides.
6. **F1** = $2PR/(P+R)$: harmonic mean, high only when both are high - a single balanced score.
7. They focus on the positive (fraud) class so they reflect real performance on imbalanced data; confusion matrix / PR-curve adds detail.

c) Metric to prioritise

8. Prioritise **recall**.
9. A missed fraud (FN) causes direct financial loss - far costlier than a false alarm needing a manual check.
10. Recall alone can be gamed by flagging everything, so also monitor precision; tune threshold or use F2.

B2. Decision Tree: 95% training, 78% testing accuracy. a) Problem? b) How does pruning help? c) One advantage of Random Forest over a single tree.

a) Problem

1. **Overfitting** (high variance).
2. The tree grows deep and memorises training noise.
3. The 17-point gap between train and test accuracy confirms it.

b) Pruning

4. Pruning removes branches with little predictive power, simplifying the tree.
5. Pre-pruning stops growth early: max_depth, min_samples_split, min_samples_leaf, minimum impurity decrease.
6. Post-pruning grows the full tree then cuts weak branches, e.g. cost-complexity pruning (ccp_alpha) chosen on validation data.
7. Smaller tree = lower variance, better generalisation, easier to interpret.

Example: limiting max_depth from 20 to 5 may drop train accuracy to 88% but raise test accuracy to 85%.

c) Random Forest vs single tree

Point	Single Decision Tree	Random Forest
8. Structure	A single decision tree is built on the whole training set.	A random forest combines many trees, each trained on a bootstrap sample with a random subset of features (bagging).
9. Variance / overfitting	A single deep tree has high variance and overfits the training data easily.	A random forest has low variance, because averaging the trees' predictions cancels out their individual errors.
10. Robustness	A single tree is sensitive to noise and small changes in the data.	A random forest is robust to noise and outliers and also gives feature importance.

B3. Working principle of SVM. a) Hyperplane b) Margin and why maximise it c) Purpose of kernel trick with an example kernel.

Working principle

1. SVM is a supervised classifier that finds the decision boundary separating the classes with the widest possible gap.

a) Hyperplane

2. A hyperplane is the decision boundary $w \cdot x + b = 0$.
3. It is a line in 2-D, a plane in 3-D, a hyperplane in higher dimensions.
4. Points with $w \cdot x + b > 0$ go to one class, < 0 to the other.

Example: in 2-D, the line $x_1 + x_2 - 5 = 0$ separates the two classes.

b) Margin

5. Margin = distance between the hyperplane and the closest points of each class; those points are the **support vectors**. Margin = $2/\|w\|$.
6. Maximising the margin means minimising $\|w\|$.
7. Larger margin → better generalisation, lower overfitting risk, robust to noise. Only support vectors define the boundary.

c) Kernel trick

8. For non-linearly separable data the kernel implicitly maps it to a higher-dimensional space where it becomes separable, without computing the mapping.
9. Example: **RBF kernel** $K(x, x') = \exp(-\gamma\|x - x'\|^2)$ handles circular/complex boundaries; others: polynomial, linear.
10. Soft-margin SVM uses parameter C to trade off margin width against misclassification.

B4. Correlated features. a) How PCA uses covariance matrix and eigen decomposition b) Explained variance c) Why keep the first few PCs.

a) Covariance matrix and eigen decomposition

1. Standardise the data (zero mean, unit variance).
2. Compute covariance matrix $C = X^T X / (n-1)$; correlated features give large off-diagonal values.
3. Eigen decomposition $C v = \lambda v$: eigenvectors = principal component directions, eigenvalues = variance along them.
4. Sort eigenvectors by eigenvalue (descending) and select the top k.

5. Project: $Z = X \cdot W_k$ – new components are uncorrelated and fewer.

Example: 10 correlated features reduced to 3 components.

b) Explained variance

6. Ratio $\lambda_i / \sum \lambda$ – share of total variance carried by each PC.

7. Cumulative sum (e.g. $\geq 95\%$) tells how many components to keep.

Example: eigenvalues 4, 1, 0.5, 0.5 $\rightarrow PC1 = 4/6 = 66.7\%$.

c) Why keep the first few PCs

8. Keep most information and remove redundancy / multicollinearity.

9. Reduce overfitting and the curse of dimensionality; speed up training.

10. Filter noise and allow 2-D / 3-D visualisation.

B5. Customer transactions. a) K-means for customer groups b) One limitation of K-means and how DBSCAN addresses it c) Silhouette Score for evaluation.

a) K-means for customer groups

1. Choose k using the elbow method or silhouette score.

2. Initialise k centroids (k-means++).

3. Assign each customer to the nearest centroid (Euclidean distance).

4. Recompute centroids as the mean of members; repeat until assignments stop changing. Scale features first.

Example: k = 3 on (annual spend, visit frequency) $\rightarrow$ high-value, regular and occasional customers.

b) Limitation and DBSCAN

Point	K-means	DBSCAN
5. Number of clusters	K-means requires the number of clusters k to be specified in advance.	DBSCAN finds the number of clusters automatically from the density of the data.
6. Cluster shape	K-means assumes clusters are roughly spherical and of similar size.	DBSCAN can find clusters of arbitrary shape.
7. Outliers	K-means is sensitive to outliers, as every point is forced into a cluster and pulls the centroids.	DBSCAN labels low-density points as noise (using eps and minPts), so outliers do not distort the clusters.

c) Silhouette score

8. $s = (b - a) / \max(a, b)$; a = mean intra-cluster distance, b = mean nearest-cluster distance.

9. Average over all points, -1 to +1: near +1 good, near 0 overlap, negative misassignment.

10. Compute for different k and pick the k with the highest score.

Example: average silhouette 0.42 (k=2), 0.61 (k=3), 0.48 (k=4) $\rightarrow$ choose k = 3.

B6. High-dimensional data with outliers. a) t-SNE / UMAP b) IQR method c) One tree-based outlier detection method.

a) t-SNE / UMAP

1. Both are non-linear dimensionality-reduction methods projecting high-dimensional data to 2-D/3-D for visualisation.

Example: 784-pixel handwritten digits plotted in 2-D show ten clusters, one per digit.

Point	t-SNE	UMAP
2. Idea	t-SNE preserves the local neighbourhoods of the data, using probability distributions and a t-distribution in the low-dimensional space.	UMAP is based on manifold learning and builds a neighbour graph, which it then lays out in low dimensions.
3. Speed and structure	t-SNE is slower on large data and preserves global structure less well.	UMAP is faster and preserves more of the global structure along with the local structure.

b) IQR method

4. Compute Q1 (25th percentile) and Q3 (75th percentile).
5. $IQR = Q3 - Q1$.
6. Lower fence = $Q1 - 1.5 \times IQR$; Upper fence = $Q3 + 1.5 \times IQR$.
7. Values outside the fences are outliers (same rule as boxplot whiskers); no normality assumption; applied per feature.

Example: $Q1 = 20$, $Q3 = 40 \rightarrow IQR = 20$, fences = -10 and 70 ; a value of 95 is an outlier.

c) Isolation Forest

8. Outliers are few and different, so they are easy to isolate.
9. Builds random trees using random features and random split values; outliers are isolated in **fewer splits** (short path length).
10. Anomaly score is based on the path length averaged over all trees – a shorter average path means the point is more likely an anomaly.

Sources cross-checked

- scikit-learn docs – silhouette_score / silhouette_samples: $s = (b - a) / \max(a, b)$, range -1 to 1 (scikit-learn.org)
- scikit-learn docs – IsolationForest: random feature + random split; outliers need fewer splits (shorter path length) (scikit-learn.org)
- scikit-learn docs – PCA: explained_variance_ratio_ = eigenvalue / sum of eigenvalues; n_components=0.95 keeps 95% variance (scikit-learn.org)
- scikit-learn docs – DBSCAN: core samples, eps, min_samples, noise label -1 ; arbitrary shapes, no k needed (scikit-learn.org)
- SMOTE (Chawla et al., 2002, arxiv.org/abs/1106.1813): $x_{\text{new}} = x + \delta(x_{\text{neighbour}} - x)$, δ random in $[0,1]$; also GeeksforGeeks “SMOTE for Imbalanced Classification”
- AIC vs BIC: penalty $2k$ vs $k \cdot \ln(n)$; BIC stricter for $n > 7$ (CASRAI comparison, standard textbooks)
- LDA vs PCA: LDA supervised, at most (classes – 1) components; PCA unsupervised (scikit-learn LDA/PCA docs and tutorials)

Note: verified via search-result summaries of these pages; the pages themselves could not be opened in this environment. Confirm wording against your class notes.